Plan de desarrollo — Organiviaje (organizador de viajes grupales)

Contexto

Ya existe el análisis completo del TP en `docs/analisis/`: modelo conceptual UML (37 clases, 4 módulos: Núcleo, Decisiones, Planificación, Finanzas), User Story Mapping (185 historias, R1=68 / R2=74 / R3=43, priorizadas por B/E), casos de uso CU1–CU4, diagrama de actividad del CU1 y guion de presentación con supuestos (grupos de 2–20, web responsive, una moneda base, acceso por invitación; fuera de alcance: pagos in-app, reservas integradas, chat general).

Objetivo: llevar ese análisis a código en el repo vacío `https://github.com/FacuAmaya/Organiviaje`, con **FastAPI + Next.js**, buenas prácticas, **una rama por feature**, trabajo en **equipo de 2–4**, avanzando **punto por punto** (se debate cada feature antes de empezarla) y con documentación viva: un `.md` por feature + docs generales (`CONTEXT.md`, `UX.md`, `AGENTS.md`). Alcance: completo (R1 → R2 → R3). Deploy: Docker local desde el día 1, deploy público al cerrar el MVP. Auth: JWT propio.

Nivel buscado: casi profesional. Cada feature se trabaja en profundidad (modelo, validaciones, errores, estados de UI, accesibilidad, tests, docs). No se aceptan "atajos de TP".

Resultado final: web app **deployada**, **instalable en el celular como PWA** (con pasos simples para instalarla en Android/iOS), responsive y sin perder calidad en cualquier dispositivo (mobile, tablet, desktop).

Entregable inmediato al aprobar: **PDF del plan** (`docs/plan/plan-desarrollo.pdf`) + ejecución solo de la Fase 0 / rama 00, y luego frenamos a debatir la siguiente.

---

1. Stack y decisiones técnicas

Capa	Elección	Por qué
Monorepo	`backend/` + `frontend/` + `docs/` en un repo	Un PR puede tocar API y UI de la misma historia
Backend	Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2 (async) + Alembic, `uv`	Tipado, OpenAPI automático, migraciones versionadas
DB	PostgreSQL 16 (+ Redis para pub/sub de tiempo real)	Relacional encaja con el modelo UML; Redis para WebSocket multi-instancia
Auth	Email+password, hash argon2, JWT access (corto) + refresh, cookies httpOnly	Se entiende el flujo completo
Tiempo real	WebSockets de FastAPI, canal por viaje, Redis pub/sub	CU3 / US183–US184: "actualizar sin recargar"
Frontend	Next.js (App Router) + TypeScript estricto, Tailwind, shadcn/ui	Estándar actual, componentes accesibles

Capa	Elección	Por qué
Datos en front	TanStack Query + cliente tipado generado desde OpenAPI (`openapi-typescript`)	Contrato único back↔front, sin tipos duplicados a mano
Formularios	react-hook-form + zod	Validación en cliente coherente con Pydantic
Mapas	MapLibre GL + tiles OpenStreetMap; geocoding con Nominatim	Gratis, sin API key; "Servicio de mapas" del diagrama de CU
Calidad back	Ruff (lint+format), mypy, pytest + httpx, coverage ≥ 80% en dominio	
Calidad front	ESLint, Prettier, Vitest + Testing Library, Playwright (e2e)	
Infra	docker-compose (db, redis, backend, frontend), GitHub Actions CI, pre-commit	
Deploy (al final)	Vercel (front) + Render/Railway (API + Postgres + Redis)	
PWA	Web App Manifest + service worker (Serwist), íconos/splash, `display: standalone`, offline básico del shell	Instalable en el celular sin tiendas
Responsive	Mobile-first, breakpoints Tailwind, imágenes optimizadas (`next/image`), safe-areas iOS, touch targets ≥ 44px	Calidad igual en cualquier device

2. Arquitectura

Backend modular por dominio: un módulo por cada módulo del UML, todos con la misma estructura en capas:

```
backend/app/
  core/          config, db, security, deps, errores
  modules/
    nucleo/        usuarios, viajes, miembros, invitaciones, ubicaciones, posiciones
    decisiones/    propuestas (+subtipos), comentarios, encuestas, votos, decisiones
    planificacion/ itinerario, días, actividades, reservas, tareas, documentos
    finanzas/      presupuesto, gastos, aportes, partes, criterios, liquidaciones
    realtime/      websocket manager + pub/sub
  cada módulo:   models.py · schemas.py · repository.py · service.py · router.py · domain.py
```

- `domain.py`: lógica pura sin DB (p. ej. `CriterioDivision` con Strategy: Igualitaria/PorMonto/PorPorcentaje/PorPartes; simplificación de deudas; cálculo de resultado de encuesta y desempate). Ahí está casi toda la lógica que vale la pena testear.
- Regla de dependencias: router → service → repository → models. Los módulos se comunican solo a través de los services de `nucleo` (equivale al «ref» del UML).
- Herencia de `Propuesta` como single-table con discriminador `tipo` (más simple para listar/votar).
- `MiembroViaje` concentra autorización: dependencia `require_member(viaje_id)` / `require_organizer(viaje_id)` (Organizador hereda de Integrante).
- Dinero en `Decimal` / `NUMERIC(12,2)`, nunca float.

Frontend por features:

```
frontend/src/
  app/(auth)/…  app/(app)/viajes/[viajeId]/{ideas,votaciones,itinerario,mapa,gastos,hoy}/
  features/<feature>/{components,hooks,api}.ts
  components/ui/   (shadcn)   lib/api/ (cliente generado)   lib/realtime/
```

3. Organización del trabajo en equipo (Git + GitHub)

- **GitHub Flow**: `main` siempre desplegable y protegida (PR obligatorio, 1 aprobación, CI verde, sin push directo).
- **Ramas en español y legibles**: `base/` (infraestructura), `funcionalidad/` (algo que el usuario puede hacer), `diseno/` (look & feel), `experimento/` (prueba de concepto), `arreglo/` (bug), `docs/` (solo documentación). Formato `tipo/NN-que-hace`, p. ej. `funcionalidad/16-registrar-y-dividir-gastos`. `NN` = número del paso en el plan.
- **Commits**: Conventional Commits (`feat(gastos): dividir gasto en partes iguales (US114)`), siempre con el ID de historia.
- **PRs**: template con checklist (historias cubiertas, tests, md de feature actualizado, capturas). Squash merge. PRs chicos: si una feature crece, se parte en sub-PRs.
- **GitHub Projects**: 1 issue por historia (US###), labels `modulo:*`, `release:R1/R2/R3`, `tipo:*`; milestones R1/R2/R3; columnas Backlog → Ready → In progress → Review → Done.
- **Definition of Ready**: historia con criterios de aceptación y feature `.md` creado. **Definition of Done**: tests pasando, CI verde, md actualizado, revisado por otro integrante, demo local OK.
- **Versionado**: SemVer, tag al cerrar cada release (`v0.1.0` fundaciones, `v1.0.0` R1, `v2.0.0` R2…), `CHANGELOG.md`.
- `CODEOWNERS` por carpeta para repartir revisión.

4. Sistema de documentación

Generales (raíz del repo, se actualizan al cerrar cada feature):

- `README.md`: qué es, cómo levantarlo (`docker compose up`), links.
- `CONTEXT.md`: dominio, glosario (Viaje, MiembroViaje, Propuesta…), mapeo módulo UML ↔ código, decisiones y supuestos, **estado global**: tabla de features (hecha / en curso / pendiente) e historias cubiertas.
- `UX.md`: principios (mobile-first), navegación, flujos principales (CU1–CU4), design tokens/componentes, estados vacíos/carga/error, accesibilidad. Es el **resultado consolidado** de `docs/ux/`.
- **`docs/ux/`**: carpeta de investigación de look & feel (ver detalle abajo).
- `AGENTS.md`: reglas para quien escribe código (humano o IA): comandos, estructura, convenciones, cómo crear una feature, qué no hacer. (Con `CLAUDE.md` que apunta a `AGENTS.md`.)
- `ARCHITECTURE.md` + `docs/adr/NNNN-titulo.md` (Architecture Decision Records: una decisión por archivo, p. ej. "0001 monorepo", "0002 JWT en cookies").
- `docs/analisis/`: copia de los 5 PDFs del TP.

Carpeta `docs/ux/` — investigación de look & feel (se llena a ritmo propio):

Ruta	Contenido
`README.md`	Cómo usar la carpeta y estado del proceso
`01-investigacion/`	Referencias, apps que gustan (Wanderlog, Splitwise…), capturas y notas
`02-moodboard/`	Colores, tipografías, íconos, estilo de ilustración
`03-propuestas/`	Ideas cosechadas: wireframes, bocetos, decisiones de estilo
`04-design-system.md`	Tokens finales (color, tipografía, espaciado, radios, sombras) y componentes

Flujo: investigás → dejás material en 01/02 → cuando tenés algo cosechado me lo pasás → debatimos → se vuelca a `04-design-system.md` + `UX.md` → rama `diseno/estilo-visual-vN` actualiza `tailwind` /tema shadcn de toda la app. Mientras tanto se usa un tema neutro provisorio, fácil de reemplazar (todo vía tokens, nada de colores hardcodeados).

Por feature: **`docs/features/NN-descripcion.md`**, primer commit de cada rama, último update antes del PR:

```
# NN · Nombre de la feature          Rama: funcionalidad/NN-descripcion   Estado: en curso
## Objetivo
## Historias cubiertas (US### + criterios de aceptación)
## Decisiones tomadas en el debate (y alternativas descartadas)
## Diseño: modelos/migraciones · endpoints · pantallas · eventos realtime
## Hecho  (checklist)
## Falta / deuda técnica / ideas para después
## Cómo probarlo (comandos, usuario de prueba, flujo manual)
```

Al mergear queda en `main`, así la historia del proyecto es legible feature por feature.

5. Ciclo de cada feature (punto por punto)

1. **Debate** (conmigo): alcance, modelo, endpoints, pantallas → se escribe el `docs/features/NN-*.md`.
2. Issues en GitHub + rama `funcionalidad/NN-descripcion` desde `main` actualizada.
3. Backend: modelo + migración Alembic → `domain.py` con tests unitarios (TDD donde hay lógica) → repository/service → router + tests de API.
4. Regenerar cliente OpenAPI en el front.
5. Frontend: hooks de datos → componentes → página → tests de componente; e2e Playwright del flujo feliz y de errores, en viewport mobile y desktop. Checklist de calidad por feature: validaciones y mensajes de error claros, estados vacío/carga/error, permisos (integrante vs organizador), accesibilidad (teclado, contraste, labels), responsive revisado en 3 tamaños, sin warnings de lint/tipos.
6. Actualizar md de feature, `CONTEXT.md` (estado), `UX.md` si hubo pantallas nuevas, `CHANGELOG.md`.
7. PR → revisión de un compañero → squash merge → cerrar issues.

6. Hoja de ruta de ramas

Fase 0 — Fundaciones (`v0.1.0`)

#	Rama	Contenido
00	`base/00-preparar-repositorio`	Estructura monorepo, docs generales iniciales (README, CONTEXT, UX, AGENTS, ARCHITECTURE, ADRs 0001–0004), templates de PR/issue, CODEOWNERS, `.editorconfig`, `.gitignore`, PDFs de análisis, PDF del plan
01	`base/01-backend-fastapi`	FastAPI + config (pydantic-settings), DB async, Alembic, `/health`, manejo de errores, pytest, Ruff, mypy
02	`base/02-frontend-nextjs`	Next.js + TS + Tailwind + shadcn, layout base, ESLint/Prettier, Vitest, Playwright
03	`base/03-docker-y-ci`	docker-compose (db, redis, api, web), GitHub Actions (lint+test back y front), pre-commit, branch protection
04	`funcionalidad/04-registro-e-inicio-de-sesion`	Usuario, registro/login/logout/refresh, JWT en cookies, `get_current_user`, páginas login/registro, middleware de rutas protegidas
05	`experimento/05-tiempo-real`	Prueba de concepto WebSocket + Redis + mapa (mayor riesgo técnico, se valida temprano y se descarta/reutiliza)

R1 — MVP (`v1.0.0`)

#	Rama	Historias
06	`funcionalidad/06-crear-y-ver-viajes`	US001, US005, US010
07	`funcionalidad/07-invitar-al-grupo`	US003, US004, US006
08	`funcionalidad/08-proponer-ideas`	US025–US030, US035, US039, US013, US020, US063, US064, US071, US072
09	`funcionalidad/09-comentar-propuestas`	US037, US151
10	`funcionalidad/10-votar-propuestas`	US041, US044, US047, US053, US054, US015 (+ regla de desempate básica)
11	`funcionalidad/11-confirmar-decisiones`	US016, US021, US056, US057, US061, US068
12	`funcionalidad/12-presupuesto-del-viaje`	US022, US141, US142, US147
13	`funcionalidad/13-armar-itinerario`	US075, US077, US078, US079, US090
14	`funcionalidad/14-mapa-del-viaje`	US029, US043, US066, US091–US093, US097, US099, US101, US109
15	`funcionalidad/15-ubicacion-en-tiempo-real`	US183, US184

#	Rama	Historias
16	`funcionalidad/16-registrar-y-dividir-gastos`	US111–US115, US118, US120
17	`funcionalidad/17-balances-y-deudas`	US131–US134, US176
18	`funcionalidad/18-agenda-de-hoy`	US167, US168
UX	`diseno/estilo-visual-vN`	Se intercala cuando traigas ideas cosechadas de `docs/ux/` (puede repetirse)
19	`funcionalidad/19-instalar-en-celular`	Manifest, service worker, íconos, prompt de instalación, Lighthouse PWA/performance ≥ 90
20	`base/20-publicar-la-app`	Vercel + Render/Railway, HTTPS, variables, migraciones en deploy, seed demo, guía `docs/instalar-en-celular.md` (Android: Chrome → "Instalar app"; iOS: Safari → Compartir → "Agregar a inicio") → tag `v1.0.0`

R2 — Extensión (`v2.0.0`), agrupado por épica (detalle se debate al llegar)

Todas con prefijo `funcionalidad/`; el nombre definitivo se ajusta en el debate de cada una.

`21-roles-grupo` (US002, 007–009) · `22-disponibilidad-fechas` (US017, 019, 023, 024) · `23-propuestas-v2` (US031, 032, 038, 040, 042) · `24-votaciones-v2` (US045, 051, 052, 055, 060) · `25-reservas` (US065, 067, 069, 087, 088, 148) · `26-puntos-encuentro` (US074, 107, 174) · `27-itinerario-v2` (US080, 082–084, 086, 100) · `28-mapa-filtros` (US094–096, 098, 108, 185) · `29-gastos-v2` (US116, 117, 121, 123, 128, 130, 135, 136, 138–140) · `30-plan-vs-real` (US143–146) · `31-notificaciones-historial` (US149, 150, 155, 156, 173; incluye push notifications de la PWA) · `32-tareas-checklist` (US157–161) · `33-documentos` (US163, 164) · `34-durante-el-viaje` (US169, 170, 172) · `35-cierre-viaje` (US175, 177–180)

R3 — Evolución (`v3.0.0`)

Igual que R2: prefijo `funcionalidad/`.

`36-archivo-duplicado` (US011, 012, 182) · `37-propuestas-extra` (US014, 018, 033, 034, 036, 089) · `38-votacion-avanzada` (US046, 048–050, 154) · `39-historial-decisiones` (US058, 059, 062) · `40-mapa-inteligente` (US085, 102–106, 110) · `41-multimoneda` (US124–126) · `42-offline` (US127, 166; amplía el service worker de la PWA) · `43-gastos-extra` (US119, 122, 129, 137, 181) · `44-notif-preferencias` (US152, 153) · resto (US070, 073, 076, 081, 162, 165, 171)

7. Archivos clave a crear en el primer paso (rama 00)

- Commit inicial en `main` y rama `base/00-preparar-repositorio`
- `README.md`, `CONTEXT.md`, `UX.md`, `AGENTS.md`, `CLAUDE.md`, `ARCHITECTURE.md`, `CHANGELOG.md`
- `docs/adr/0001-monorepo.md`, `0002-fastapi-nextjs.md`, `0003-jwt-cookies.md`, `0004-github-flow.md`
- `docs/features/_TEMPLATE.md`, `docs/features/00-preparar-repositorio.md`

- `docs/ux/README.md` + subcarpetas `01-investigacion/`, `02-moodboard/`, `03-propuestas/` (con `.gitkeep`) y `04-design-system.md` vacío con estructura
- `docs/analisis/*.pdf` (copia de los 5 PDFs), `docs/plan/plan-desarrollo.md` + `.pdf`
- `.github/pull_request_template.md`, `.github/ISSUE_TEMPLATE/user-story.yml`, `.github/CODEOWNERS`
- `.gitignore`, `.editorconfig`

El PDF se genera desde este plan en markdown (pandoc si está instalado; si no, script Python con `markdown` + `weasyprint` / `reportlab` en el scratchpad).

8. Verificación

- **Rama 00**: el PDF abre y se lee bien; `git log` muestra commits convencionales; PR abierto en GitHub con el template; docs enlazados desde README.
- **Fundaciones (01–03)**: `docker compose up` levanta todo; `GET /health` 200; front carga en `localhost:3000`; CI verde en el PR.
- **Cada feature**: `pytest` + `vitest` + `playwright` en verde en CI; flujo manual descripto en "Cómo probarlo" del md; criterios de aceptación de cada US chequeados.
- **PWA / multi-device**: Lighthouse (PWA, performance, accesibilidad ≥ 90); instalación probada en un Android y un iPhone reales; Playwright con viewports mobile/tablet/desktop.
- **Release**: demo end-to-end de un viaje (crear → invitar → proponer → votar → decidir → itinerario → mapa con ubicación en vivo en 2 navegadores → gasto → balance) antes de taggear `v1.0.0`.

9. Próximo paso tras aprobar

1. Generar el PDF del plan.
2. Ejecutar solo la rama `base/00-preparar-repositorio` (push y PR solo con tu confirmación).
3. Frenar y debatir la rama 01 antes de escribir código.